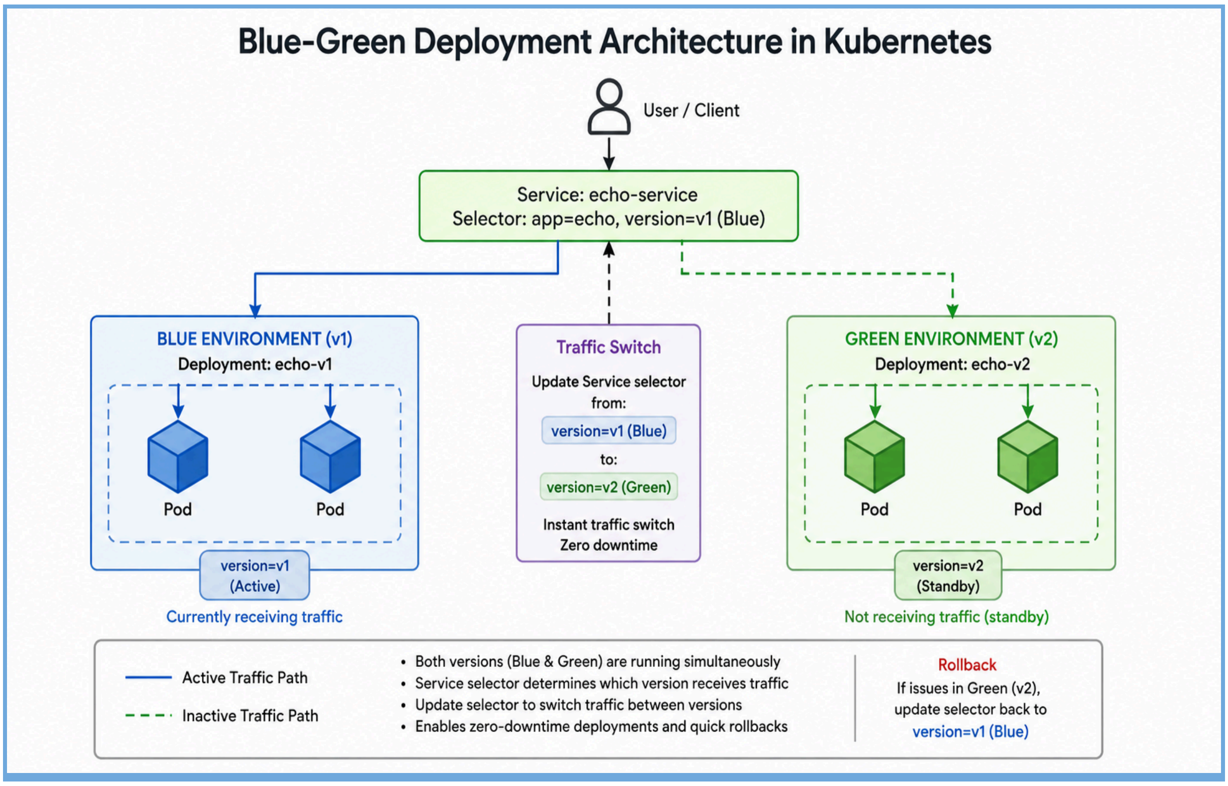
kubernetes

Blue(v1)-Green(v2) Deployment in Kubernetes



Blue-Green Deployment is a release strategy where two identical environments (v1 and v2) run simultaneously in Kubernetes. One version (Blue) serves live traffic while the other (Green) is tested, and traffic is switched instantly using a Service selector. This approach ensures **zero downtime deployments** and allows quick rollback if issues occur.

~ Girish Gajbhare

Step 1: vi ver1.yml (Blue)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: echo-v1
  labels:
    app: echo
    version: v1
spec:
  replicas: 2
  selector:
    matchLabels:
      app: echo
      version: v1
  template:
    metadata:
      labels:
        app: echo
        version: v1
      annotations:
        # Using the change-cause we discussed earlier
        kubernetes.io/change-cause: "Initial deployment of version one"
    spec:
      containers:
      - name: echo-container
        image: hashicorp/http-echo
        args:
          - "-text=hello from version one"
          - "-listen=:8080"
        ports:
        - containerPort: 8080

~
```

Step 2: Deploy Version 1 (Blue)

```
root@controlplane:~$ kubectl apply -f ver1.yml
deployment.apps/echo-v1 created
root@controlplane:~$ kubectl get pods
NAME                       READY   STATUS    RESTARTS   AGE
echo-v1-74897dd6c6-s662n   1/1     Running   0          4s
echo-v1-74897dd6c6-zrhbs   1/1     Running   0          4s
```

Step 3: Expose v1 using Service

```
root@controlplane:~$ kubectl expose deployment echo-v1 --port=8080
service/echo-v1 exposed
root@controlplane:~$ kubectl get svc
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
echo-v1      ClusterIP   10.110.8.5    <none>        8080/TCP   9s
kubernetes   ClusterIP   10.96.0.1     <none>        443/TCP    6m46s
root@controlplane:~$
```

Test:

```
root@controlplane:~$ curl 10.110.8.5:8080
hello from version one
```

Step 4: For Version 2 (Green)

Vi ver2.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: echo-v2
  labels:
    app: echo
    version: v2
spec:
  replicas: 2
  selector:
    matchLabels:
      app: echo
      version: v2
  template:
    metadata:
      labels:
        app: echo
        version: v2
      annotations:
        # Using the change-cause we discussed earlier
        kubernetes.io/change-cause: "Initial deployment of version one"
    spec:
      containers:
      - name: echo-container
        image: hashicorp/http-echo
        args:
          - "-text=hello from version two"
          - "-listen=:8080"
        ports:
        - containerPort: 8080

~
~
```

~ Girish Gajbhare

kubernetes

Step 5: Deploy Version 2 (Green)

```
root@controlplane:~$ kubectl apply -f ver2.yml
deployment.apps/echo-v2 created
root@controlplane:~$ kubectl get pods
NAME                       READY   STATUS    RESTARTS   AGE
echo-v1-74897dd6c6-s662n   1/1     Running   0          10m
echo-v1-74897dd6c6-zrhbs   1/1     Running   0          10m
echo-v2-6f56f679f5-5dpkz   1/1     Running   0          7s
echo-v2-6f56f679f5-k58kf   1/1     Running   0          7s
root@controlplane:~$
```

Step 6: Test v2

```
root@controlplane:~$ kubectl get pods -o wide
NAME                       READY   STATUS    RESTARTS   AGE    IP              NODE           NOMINATED NODE   READINESS GATES
echo-v1-74897dd6c6-s662n   1/1     Running   0          12m    192.168.1.169   node01         <none>           <none>
echo-v1-74897dd6c6-zrhbs   1/1     Running   0          12m    192.168.0.53    controlplane   <none>           <none>
echo-v2-6f56f679f5-5dpkz   1/1     Running   0          103s   192.168.0.165   controlplane   <none>           <none>
echo-v2-6f56f679f5-k58kf   1/1     Running   0          103s   192.168.1.206   node01         <none>           <none>
root@controlplane:~$ curl 192.168.1.206:8080
hello from version two
root@controlplane:~$
```

Step 7: Verify Both Versions

```
root@controlplane:~$ kubectl get pods --show-labels
NAME                       READY   STATUS    RESTARTS   AGE     LABELS
echo-v1-74897dd6c6-s662n   1/1     Running   0          13m     app=echo,pod-template-hash=74897dd6c6,version=v1
echo-v1-74897dd6c6-zrhbs   1/1     Running   0          13m     app=echo,pod-template-hash=74897dd6c6,version=v1
echo-v2-6f56f679f5-5dpkz   1/1     Running   0          3m15s   app=echo,pod-template-hash=6f56f679f5,version=v2
echo-v2-6f56f679f5-k58kf   1/1     Running   0          3m15s   app=echo,pod-template-hash=6f56f679f5,version=v2
root@controlplane:~$
```

You should see:

- v1 pods → `version=v1`
- v2 pods → `version=v2`

Step 8: Switch Traffic (Blue → Green)

Change selector from version=v1 to version=v2 of svc echo-v1

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: "2026-05-01T04:53:08Z"
  labels:
    app: echo
    version: v1
  name: echo-v1
  namespace: default
  resourceVersion: "11014"
  uid: 9b7bde03-1138-470c-98a0-b3ff782c6c90
spec:
  clusterIP: 10.110.8.5
  clusterIPs:
  - 10.110.8.5
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
    app: echo
    version: v2
  sessionAffinity: None
  type: ClusterIP
status:
  loadBalancer: {}
~
~
```

```
root@controlplane:~$ kubectl edit svc echo-v1
service/echo-v1 edited
root@controlplane:~$
```

Step 9: Test After Switching

```
root@controlplane:~$ curl 10.110.8.5:8080
hello from version two
root@controlplane:~$
```

Step 10: Rollback to Blue

Edit service again:

```
  selector:
    app: echo
    version: v1
```

Verify Traffic Switched

```
root@controlplane:~$ curl 10.110.8.5:8080
hello from version one
root@controlplane:~$
```

💥 Traffic instantly goes back to old version

Conclusion:

Blue-Green deployment in Kubernetes provides a reliable way to release applications with **zero downtime and minimal risk**. By running two versions simultaneously and switching traffic using a Service selector, you get instant deployment and easy rollback. Overall, it improves system stability, user experience, and confidence in production releases.

~ Girish Gajbhare